

BACHELOR STUDENT TOOLKIT

Python Mid-Term Project Report

Project Title	Bachelor Student Toolkit
Module Covered in This Report	Student Profile & Academic Records (my individually completed part)
Student Name	Alim Al Razi Shatak
Student ID	23-51897-2
Course / Section	Programming in Python [Section: D]
Instructor	MD. TANZEEM RAHAT
Date of Submission	25 July 2026

Note: This is a group project split into four modules (Student Profile & Academic Records, Class Routine & Exam Schedule, Expense Tracker, Study Planner). This report documents only the Student Profile & Academic Records module, which is complete. The remaining three modules are being developed by teammates and are marked as PENDING throughout this report; sections describing them will be completed once those modules are finished.

1. Project Title and Group / Member Information

Project Title: Bachelor Student Toolkit — a console-based application that helps a bachelor's-degree student manage their academic profile and records in one place.

Module	Responsible Member	Status
Student Profile & Academic Records	Alim Al Razi Shatak (23-51897-2)	Complete
Class Routine & Exam Schedule	[Teammate name — pending]	Pending
Expense Tracker	[Teammate name — pending]	Pending
Study Planner	[Teammate name — pending]	Pending

2. Problem Statement and Objectives

Bachelor students typically track their personal information, semester course loads, marks, and GPA manually across spreadsheets or paper, which is error-prone and hard to maintain over several semesters. This project builds a single console application where a student's profile and academic history can be registered, retrieved, updated, and analyzed consistently, with all data persisted between runs.

Objectives of the Student Profile & Academic Records module:

- Allow a student to register a profile once and retrieve/update/delete it later using a unique Student ID.
- Allow marks for each course, in each semester, to be entered incrementally across multiple sessions without overwriting previously saved semesters.
- Automatically calculate letter grade, grade point, semester GPA, and overall CGPA from stored marks.
- Provide an administrator-only dashboard summarizing CGPA statistics across all registered students using NumPy.
- Persist all data in CSV files so no information is lost when the program closes.

3. Feature List and Target Users

Target users:

(1) a bachelor student maintaining their own profile and academic record, and (2) an administrator (e.g. course coordinator) who needs an aggregate view of all students via a passkey-protected dashboard.

3.1 Implemented Features — Student Profile & Academic Records (this report)

#	Feature	Description
1	Register Student	Collects and validates Student ID, name, university, department, and email; rejects duplicate IDs.
2	View Student Profile	Looks up and displays a student's saved profile by ID.
3	Update Student Profile	Edits one or more fields, re-validating each changed field; unchanged fields are kept.
4	Delete Student Profile	Removes a student's profile after confirmation.
5	View All Students (admin)	Passkey-protected listing of every registered student.
6	Add Semester Courses (chained)	Adds course records semester by semester; detects previously saved semesters and continues numbering instead of overwriting them.
7	View Semester Courses	Displays all courses recorded for a specific semester.
8	Calculate Semester GPA	Computes credit-weighted GPA for one semester.
9	Calculate Overall CGPA	Computes credit-weighted CGPA across all recorded semesters.

10	View Transcript	Prints a full semester-by-semester transcript with running and overall GPA.
11	Dashboard (admin)	NumPy-based mean, median, standard deviation, minimum, and maximum CGPA across all students.

3.2 Pending Features (Other Modules — Not Covered in This Report)

Module	Status	Notes
Class Routine & Exam Schedule	Pending	Menu option exists in main.py but currently prints "under development".
Expense Tracker	Pending	Menu option exists in main.py but currently prints "under development".
Study Planner	Pending	Menu option exists in main.py but currently prints "under development".

4. Explanation of Used Python Concepts (Week 1–5)

Concept	Where It Is Used in the Student Profile Module
Variables & data types	student_id, full_name, marks (float), credit (int), letter_grade (str) all stored and passed between functions in manager.py and storage.py.
Operators	Comparison operators validate ranges ($0 \leq \text{marks} \leq 100$); arithmetic operators compute $\text{total_grade_points} += \text{credit} * \text{grade_point}$ for GPA/CGPA.
Branching (if / elif / else)	Used throughout profile_menu() and academic_records_menu() to route menu choices, and inside manager.py to validate input and check whether a student/course record exists.
Loops (while / for)	while loops re-prompt on invalid input (e.g. student ID format); for loops iterate over semesters, course counts, and stored CSV rows when computing GPA/CGPA.
Functions	Each operation (register_student, view_student, update_student, delete_student, add_semester_courses, calculate_semester_gpa, calculate_overall_cgpa, view_transcript, dashboard_statistics) is a separate function with a single responsibility.

5. Data Structure Explanation: List, Tuple, Set, Dictionary

Structure	Example in Code	Purpose
List	<code>students = []</code> in <code>get_all_students()</code> ; <code>courses.append(row)</code> while reading <code>courses.csv</code>	Collects multiple student/course records read from file for iteration and display.
Tuple	<code>GRADE_RANGES = {"A+": (90, 100), "A": (85, 89.99), ...}</code> in <code>grading.py</code>	Stores the fixed (minimum, maximum) mark boundaries for each letter grade — an immutable pair per grade.
Set	<code>semesters = set()</code> in <code>calculate_overall_cgpa()</code> ; <code>used_course_codes = set()</code> in <code>add_semester_courses()</code>	Tracks unique semester numbers for the completed-semester count, and prevents entering the same course code twice within one semester.
Dictionary	Each CSV row is read via <code>csv.DictReader</code> , giving <code>row["student_id"]</code> , <code>row["full_name"]</code> , etc.	Represents one student or course record as key-value pairs mapped directly to CSV column names.

6. OOP Design: Class Name, Attributes, and Methods

The module follows a data-model-class + manager-class structure recommended in the project brief: Student and Course represent single records, while `manager.py` contains the application/manager logic that operates on collections of these records.

6.1 Student class (`modules/profile/student.py`)

Attribute	Type	Description
<code>student_id</code>	<code>str</code>	Unique 5-digit identifier
<code>full_name</code>	<code>str</code>	Student's full name
<code>university</code>	<code>str</code>	University name
<code>department</code>	<code>str</code>	Department name
<code>email</code>	<code>str</code>	Contact email

Method: `display_info()` — prints a formatted summary of the student's profile.

6.2 Course class (modules/profile/course.py)

Attribute	Type	Description
student_id	str	ID of the student the course belongs to
semester	int	Semester number
course_code	str	Course code, e.g. CSE101
course_name	str	Course title
credit	int	Credit hours (1–4)
marks	float	Marks obtained (0–100)
letter_grade	str	Computed letter grade
grade_point	float	Computed grade point

6.3 Manager layer (modules/profile/manager.py)

manager.py is the application/manager layer: it does not represent a single entity, but coordinates registration, validation, storage calls, and GPA/CGPA/transcript calculations across many Student and Course objects — matching the brief's recommendation to separate a data model class from a manager/application class.

7. File Handling and Exception Handling Explanation

All student and course data is stored in two CSV files, data/students.csv and data/courses.csv, accessed only through modules/profile/storage.py.

- Save feature: save_student() and save_course() append a new row to the relevant CSV, writing a header row first if the file is new or empty.
- Load feature: get_student(), get_all_students(), get_semester_courses(), get_all_courses(), and get_all_courses_records() read rows back using csv.DictReader.
- Missing file handling: an ensure_file() helper checks os.path.exists() before every read or write and creates the file (with the correct header row) automatically if it is missing, so the program never crashes on first run.
- Corrupted / incomplete row handling: dictionary access uses row.get("field") instead of row["field"] in read paths, and every place that converts a stored value to a number (e.g. float(course["credit"])) is wrapped in try/except (KeyError, ValueError, TypeError), so one malformed row is skipped with a message instead of stopping the whole calculation.
- Input exception handling: marks entry uses try/except ValueError to reject non-numeric input and re-prompt the user without crashing.
- Duplicate prevention: register_student() calls get_student() before saving and rejects an ID that is already registered, looping the user back to enter a different one.

- Divide-by-zero protection: GPA/CGPA calculations check that total credits are greater than zero before dividing.

8. Library / Module Use: NumPy

NumPy is used in `dashboard_statistics()` (admin-only, passkey-protected) to compute descriptive statistics over every student's CGPA:

```
cgpa_array = np.array(cgpa_list)
np.mean(cgpa_array) # Average CGPA across all students
np.max(cgpa_array)  # Highest CGPA
np.min(cgpa_array)  # Lowest CGPA
np.median(cgpa_array) # Median CGPA
np.std(cgpa_array)   # Standard deviation of CGPA
```

This satisfies the project's external-library requirement with a real analytical purpose rather than a decorative import.

9. Screenshots and Sample Input/Output

Replace the console excerpt below with actual screenshots of the running application before final submission.

Sample session (Register → Add Semester Courses, chained across two sessions → Semester GPA → Overall CGPA → Transcript → Admin Dashboard):

```
===== Register a new student =====
Enter Student ID: 23519
Enter Full Name: Alim Al Razi
...
===== Student registered successfully! =====

===== Add Semester Courses =====
No previous records found. Starting from Semester 1.
... (Semester 1: CSE101, Semester 2: CSE102 saved)

===== Add Semester Courses ===== (second run)
===== Previous Records Found =====
Semesters already recorded : 1, 2
  CSE101 - Intro to CS (Credit: 3, Grade: A)
  CSE102 - Data Structures (Credit: 3, Grade: B)
New semesters will continue from Semester 3.
... (Semester 3: CSE201 saved — no overwrite of 1 or 2)

===== Overall CGPA =====
Completed Semesters      : 3
Total Courses            : 3
Overall CGPA             : 3.67

===== Dashboard =====
Average CGPA             : 3.67
Highest CGPA             : 3.67
Standard Deviation       : 0.00
```

10. Limitations and Possible Future Improvements

Limitations:

- Interface is console/menu-based only; Class Routine, Expense Tracker, and Study Planner modules are still pending from teammates, so the toolkit is not yet feature-complete as a whole.
- Name/university/department validation only accepts letters and spaces, so legitimate names containing hyphens or apostrophes would currently be rejected.
- Course-code duplicate prevention only applies within a single semester-entry session, not across a student's entire history.
- Admin passkey is stored as a plain constant in utils/config.py rather than an environment variable or hashed value.

Possible future improvements:

- Add a Tkinter interface once the other three modules are complete, to satisfy the GUI-oriented mark distribution row more strongly.
- Allow apostrophes/hyphens in name-style fields.
- Extend duplicate-course-code checking across a student's full transcript, not just the current session.
- Integrate all four modules behind one shared main.py dashboard once teammates finish their parts.